

# RAG Foundations

## *Loading Documents, Chunking Strategies, Metadata, and Web Scraping*

Naresh Chaurasia

Version 1.0, 2026-07-03: First Edition

# Table of Contents

|                                                    |    |
|----------------------------------------------------|----|
| 1. The Problem                                     | 1  |
| 2. The Concept                                     | 2  |
| 3. How It Works                                    | 3  |
| 3.1. The FreightPath Pipeline                      | 3  |
| 3.2. Stage 1: Loading                              | 3  |
| 3.3. Stage 2: Chunking                             | 4  |
| 3.4. Stage 3: Metadata Tagging                     | 4  |
| 3.5. Stage 4: Embedding and Storage                | 4  |
| 4. Key Components                                  | 5  |
| 4.1. Loading Documents from Multiple Sources       | 5  |
| 4.1.1. PDF                                         | 5  |
| 4.1.2. DOCX                                        | 5  |
| 4.1.3. HTML                                        | 5  |
| 4.1.4. Markdown                                    | 6  |
| 4.1.5. CSV                                         | 6  |
| 4.1.6. Websites                                    | 6  |
| 4.2. Chunking Strategies                           | 6  |
| 4.2.1. Fixed-Size Chunking                         | 7  |
| 4.2.2. Sentence-Based Chunking                     | 7  |
| 4.2.3. Recursive Chunking                          | 7  |
| 4.2.4. Comparison of Chunking Strategies           | 8  |
| 4.3. Adding Metadata to Chunks                     | 8  |
| 4.3.1. Why Metadata Matters for Retrieval Accuracy | 8  |
| 4.3.2. Why Metadata Matters for Traceability       | 8  |
| 4.3.3. Standard Metadata Fields                    | 8  |
| 4.4. Scraping Web Content for RAG                  | 9  |
| 4.4.1. Step 1: Fetching                            | 9  |
| 4.4.2. Step 2: Cleaning                            | 9  |
| 4.4.3. From Raw HTML to Clean, Chunked Text        | 10 |
| 4.4.4. Politeness and Rate Limiting                | 10 |
| 4.4.5. Web-Specific Metadata                       | 11 |
| 5. How It Solves the Problem                       | 12 |
| 6. Summary                                         | 13 |
| 7. Quick Self-Check Questions                      | 14 |
| 7.1. Foundational Questions                        | 14 |
| 7.2. Intermediate Questions                        | 14 |

# Chapter 1. The Problem

Imagine a team of engineers at a mid-sized logistics company called FreightPath. Over ten years, their operations team has accumulated roughly 8,000 documents: PDF safety manuals, DOCX carrier agreements, HTML rate sheets published by freight partners, Markdown internal wikis, CSV fuel surcharge tables, and hundreds of web pages from regulatory bodies.

A new hire named Priya joins the compliance team. On her first day, she needs to answer a seemingly simple question: "What is the maximum payload for a Class 7 truck under the updated 2023 federal guidelines, and how does our current carrier agreement with Apex Logistics handle overweight penalties?"

Without a knowledge system, Priya must manually search through thousands of files. She opens PDFs one by one, searches DOCX contracts with Ctrl+F, and navigates government websites. The answer she needs is split across at least four different documents in three different formats. It takes her four hours.

Now multiply Priya by 40 compliance officers, each spending similar time every day. That is 160 hours per day lost to document search. The cost is real: delayed decisions, regulatory risk, and exhausted staff.

This is the exact problem that Retrieval-Augmented Generation (RAG) is designed to solve. But before any intelligent retrieval can happen, the raw documents must be loaded, cleaned, broken into usable pieces, tagged with context, and stored. That preparation pipeline is the subject of this chapter.

---

# Chapter 2. The Concept

Think of building a RAG knowledge base the way a librarian organizes a large archive. When a shipment of books arrives, the librarian does not stack them in a pile and tell visitors to dig through. Instead, the librarian opens each box (loading), cuts long reference books into readable sections (chunking), writes index cards for each section noting the book title, author, chapter, and page number (metadata), and files everything in a searchable catalog.

When a visitor asks a question, the librarian checks the catalog and pulls only the relevant sections rather than handing over entire volumes.

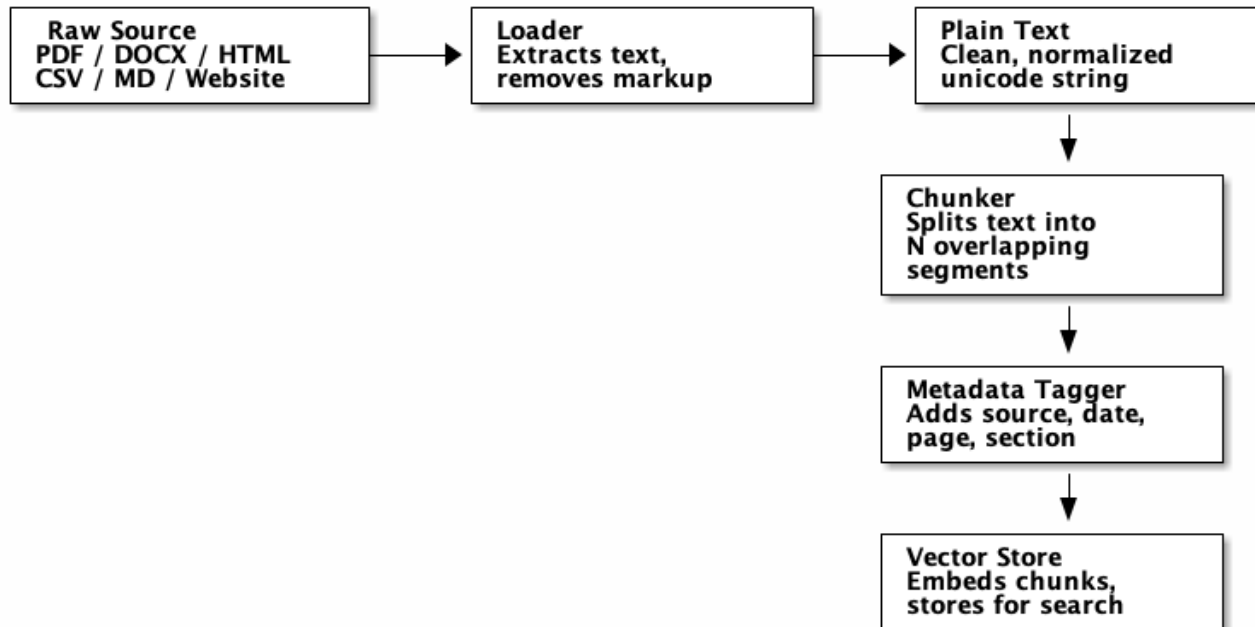
In RAG terminology:

- **Document loading** is opening the boxes—reading content out of PDFs, DOCX files, HTML pages, CSVs, Markdown files, and live websites into plain text your system can process.
  - **Chunking** is cutting the content into sections of a manageable size—small enough for an AI model to read efficiently, large enough to carry complete thoughts.
  - **Metadata** is the index card—the extra labels attached to each chunk so the retrieval system knows where it came from and what it is about.
  - **Web scraping** is the process of fetching content from live websites, cleaning away the HTML structure, and treating it like any other document source.
-

# Chapter 3. How It Works

## 3.1. The FreightPath Pipeline

FreightPath’s engineering team decides to build a RAG pipeline. The journey of a single document — say, the PDF of the federal Class 7 truck payload guidelines — illustrates every stage.



The pipeline has four stages. Each stage transforms the data into a form the next stage can use. Nothing reaches the vector store without passing through all four.

## 3.2. Stage 1: Loading

The loader’s job is deceptively simple: produce a clean plain-text string from whatever format the source document uses. In practice, this is the messiest part of the pipeline because every format stores text differently.

A PDF encodes text as a stream of character positions. A DOCX file is actually a ZIP archive containing XML. An HTML page mixes content with navigation menus, cookie banners, and JavaScript. A CSV stores structured rows with delimiters. Markdown uses symbols like asterisks and hash signs to express formatting. Each format requires a different parsing strategy.

The loader for FreightPath’s federal guidelines PDF opens the binary file, locates the text content stream, decodes character encodings, and outputs the payload table as a plain string. All formatting instructions in the PDF (fonts, colors, column widths) are discarded. Only the words and numbers survive.

### 3.3. Stage 2: Chunking

The plain text output from the loader is typically too large to use as a single unit. An AI embedding model has a context limit — often between 512 and 8192 tokens. The federal guidelines PDF might be 120 pages and 80,000 words. That cannot be embedded as one piece, nor should it be: a chunk containing everything is no more useful for retrieval than the original document.

Chunking divides the long text into shorter segments. The choice of how to divide has significant consequences for retrieval quality, which is explored in the next section.

### 3.4. Stage 3: Metadata Tagging

After chunking, each segment is labeled. The label records at minimum: the original file name, the format (PDF, DOCX, etc.), the page or line number, and the date the document was ingested. Richer labels might include the document section heading, the author, or a document category.

These labels travel with every chunk into the vector store. When retrieval happens later, the system can filter by label before searching by meaning—for example, "only search chunks from documents tagged as federal regulatory guidelines, published after 2022."

### 3.5. Stage 4: Embedding and Storage

Each labeled chunk is converted into a vector—a list of numbers that represents the chunk's meaning in a high-dimensional space. Chunks with similar meanings have vectors that are numerically close. The vector store indexes these vectors so that at query time, a question can be converted to a vector and the nearest matching chunks can be retrieved in milliseconds.

This chapter focuses on stages one through three. Embedding and vector storage are covered separately.

---

# Chapter 4. Key Components

## 4.1. Loading Documents from Multiple Sources

### 4.1.1. PDF

PDFs are the most common enterprise document format. They present two distinct challenges. A natively digital PDF (created by a word processor or typesetting tool) contains actual text that a parser can extract directly. A scanned PDF (photographed paper run through a scanner) contains only images of text; extraction requires optical character recognition (OCR) to convert pixel patterns back into characters.

For FreightPath's regulatory PDFs, which are natively digital government publications, direct extraction works. For older carrier contracts scanned from paper originals, OCR is necessary and introduces a small error rate—characters like "0" and "O" or "1" and "l" are sometimes confused, which can matter for numeric fields like payload limits.

Tools such as PyMuPDF and pdfplumber handle natively digital PDFs well. Tesseract OCR combined with pdf2image handles scanned documents. The pipeline must detect which type it is processing and route accordingly.

### 4.1.2. DOCX

DOCX files are internally structured XML. A Word document stores paragraph text, heading levels, table cells, bold/italic markers, embedded images, and comments all in separate XML tags within a ZIP archive. A DOCX loader must navigate this XML to extract text in reading order, preserving paragraph breaks while discarding formatting codes.

FreightPath's carrier agreements are DOCX files. The contract for Apex Logistics is 45 pages. The loader extracts 14,000 words of text—the exact payload penalty clauses Priya needs are in paragraph 8.4.2 of the agreement.

One complication: DOCX tables store content in a grid structure. If the loader reads cells left-to-right, top-to-bottom without understanding the table structure, a rate schedule table may become a meaningless stream of numbers. Good DOCX loaders preserve table semantics by joining row cells with a delimiter so the resulting text reads "Column A: value | Column B: value" rather than "value value value."

### 4.1.3. HTML

HTML pages mix content with navigation, advertising, and structural markup. A raw HTML loader that extracts all text would produce fragments like "Home About Contact Subscribe to our newsletter Click here Read More" mixed into the actual content.

Effective HTML loading requires a two-step approach. First, the HTML is parsed to build a document tree. Second, a content extraction heuristic is applied—this might be as simple as selecting only the content within a `<main>` or `<article>` tag, or as sophisticated as a density-based algorithm (like the one used in tools such as Trafilatura or Readability) that identifies the area of

the page with the highest ratio of text to markup.

FreightPath pulls HTML rate sheets from three carrier web portals. These pages have consistent structure: rates live inside a `<div class="rate-table">` element. The loader is configured to extract only that div, discarding the navigation and footer.

#### 4.1.4. Markdown

Markdown is the simplest format to load. The symbols used for formatting (hash signs for headings, asterisks for emphasis, backticks for code) are regular ASCII characters that can either be stripped or converted to plain text with a straightforward substitution pass.

FreightPath's internal operations wiki is written in Markdown and stored in a Git repository. The loader strips heading markers and list bullets, preserving the text content. Because Markdown headings are semantically meaningful, a good loader records the current heading hierarchy as it processes the file — this becomes valuable metadata (the section name) attached to every chunk derived from that section.

#### 4.1.5. CSV

CSV files require a fundamentally different loading strategy compared to prose documents. A CSV contains structured, tabular data. Treating a row as a text string — "Class 7, 26001, 33000, federal" — is technically possible but produces chunks that are meaningless without context about what each column represents.

The correct approach for CSV loading is to convert each row into a sentence that includes the column headers: "Truck class: Class 7. Minimum weight: 26,001 lbs. Maximum payload: 33,000 lbs. Regulation: federal." This produces text that is self-contained and semantically coherent. When retrieved, the chunk answers the question without requiring the reader to know which column was which.

FreightPath has a 3,000-row CSV of fuel surcharge rates indexed by origin region, destination region, and fuel price band. Converting each row to a sentence produces 3,000 small, highly specific chunks — each one directly answerable to a pricing query.

#### 4.1.6. Websites

Loading content from live websites is covered in detail in the web scraping section below.

## 4.2. Chunking Strategies

Chunking is where engineering judgment matters most. A chunk that is too small loses context — a single sentence may be unintelligible without the surrounding paragraph. A chunk that is too large dilutes relevance — if the top-scoring chunk at retrieval time is 2,000 words, the answer to the question may be buried deep within it, and the model has to read much more than necessary.

Three strategies dominate practical RAG systems.



### 4.2.1. Fixed-Size Chunking

Fixed-size chunking splits the text every N tokens regardless of sentence or paragraph boundaries. If N is 512 and the 513th token falls in the middle of the word "transportation," the chunk ends after "transpor" and the next chunk begins with "tation."

This boundary problem is addressed with an **overlap** parameter. Instead of clean, non-overlapping cuts, each chunk shares a trailing segment with the beginning of the next chunk. A 512-token chunk with 64-token overlap means each chunk's last 64 tokens are repeated as the first 64 tokens of the following chunk. This ensures that a sentence split across a boundary appears complete in at least one chunk.

Fixed-size chunking is the fastest strategy to implement and produces uniform chunk sizes, which simplifies embedding and storage. Its weakness is semantic blindness: it has no awareness of whether a cut happens inside a sentence, between a question and its answer, or in the middle of a numbered list.

For FreightPath's use case, fixed-size chunking is applied to the CSV-derived sentences (which are already atomic) and to large undifferentiated text bodies where structural boundaries are not meaningful.

### 4.2.2. Sentence-Based Chunking

Sentence-based chunking uses a sentence boundary detector to split text only at the end of complete sentences. The system accumulates sentences until the running token count approaches a maximum, then closes the chunk and begins a new one. Overlap is implemented by carrying the last one or two sentences of the previous chunk forward.

This strategy preserves semantic integrity at the local level. A sentence about payload limits will never be split mid-thought. The trade-off is variable chunk size — a paragraph containing one very long sentence and ten short ones will produce chunks of irregular length, which can complicate downstream processing.

Sentence chunking works well for FreightPath's regulatory PDF text, where each sentence in the legal guidelines is a complete, self-contained statement. The federal payload rules read: "A Class 7 vehicle operating under federal jurisdiction may not exceed a gross vehicle weight of 33,000 pounds on any interstate route." This sentence, kept whole, retrieves correctly when Priya asks about payload limits.

### 4.2.3. Recursive Chunking

Recursive chunking attempts to split text at the most meaningful available boundary given a target size. It tries paragraph breaks first. If a paragraph is still too large, it tries sentence breaks within that paragraph. If a sentence is too long, it falls back to word boundaries.

The "recursive" name comes from the hierarchical fallback logic: the splitter tries the highest-level structure first and recurses into finer granularity only when necessary.

This is the most context-preserving strategy and is generally recommended for documents with clear structure such as technical manuals, legal agreements, and annotated wikis. For FreightPath,

recursive chunking is applied to the DOCX carrier agreements, where the document is organized into numbered clauses and sub-clauses. A chunk derived from clause 8.4.2 stays within that clause rather than bleeding into 8.4.3.

#### 4.2.4. Comparison of Chunking Strategies

| Strategy       | Best For                                          | Main Advantage                     | Main Trade-Off                                          |
|----------------|---------------------------------------------------|------------------------------------|---------------------------------------------------------|
| Fixed-size     | Large undifferentiated text, CSVs, fast pipelines | Uniform size, simple to implement  | May split mid-sentence or mid-thought                   |
| Sentence-based | Legal text, regulatory documents, clear prose     | Preserves sentence integrity       | Variable chunk size, slower than fixed                  |
| Recursive      | Structured documents with headings and clauses    | Follows natural document structure | Most complex to configure, depends on structure quality |

### 4.3. Adding Metadata to Chunks

Every chunk that exits the chunking stage must carry a metadata payload. This is not optional overhead—it is what transforms a collection of anonymous text fragments into a traceable, filterable knowledge base.

#### 4.3.1. Why Metadata Matters for Retrieval Accuracy

Consider two chunks that contain the phrase "maximum payload is 33,000 pounds." One comes from the 2023 federal guidelines. The other comes from a 2017 state-level regulation for California only. Without metadata, the retrieval system cannot distinguish them. Priya might receive the California 2017 figure when she asked about the current federal rule.

With metadata, the retrieval system can apply a filter: "only return chunks where `source_type` equals `federal_regulation` AND `year` is greater than or equal to 2022." The 2017 California chunk is excluded before semantic search even runs.

#### 4.3.2. Why Metadata Matters for Traceability

In regulated industries — logistics, healthcare, finance, legal — a user who receives an AI-generated answer needs to know where that answer came from. If the answer says "the maximum payload is 33,000 pounds," the compliance team must be able to click through to the exact page of the exact document that contains that figure.

Metadata enables this. Each chunk carries its source file name, page number, section heading, and ingestion date. When the chunk is retrieved and used to generate an answer, the metadata is surfaced to the user as a citation. Priya sees: "Source: `federal_class7_guidelines_2023.pdf`, Page 14, Section 3.2."

#### 4.3.3. Standard Metadata Fields

At a minimum, every chunk should carry:

- **source\_file**: the original file name or URL
- **source\_format**: PDF, DOCX, HTML, CSV, Markdown, or web
- **ingestion\_date**: when the document was loaded into the pipeline
- **chunk\_index**: the position of this chunk within the source document (useful for reconstructing context)
- **page\_number** (for PDFs and DOCX): the physical page from which the chunk was extracted

For structured documents, additional fields are highly valuable:

- **section\_heading**: the heading or clause number that governs this chunk's position in the document
- **document\_category**: a label such as "federal\_regulation", "carrier\_agreement", or "internal\_policy"
- **document\_date**: the date the source document was published or last revised (distinct from the ingestion date)
- **author** or **issuing\_body**: who produced the document

For CSV-derived chunks:

- **row\_index**: the original row number in the CSV
- **key\_fields**: a summary of the primary key columns (e.g., "origin=Chicago, destination=Dallas, fuel\_band=2.50-3.00")

## 4.4. Scraping Web Content for RAG

Web scraping for RAG follows the same four-stage pipeline as file-based loading, but adds two preliminary steps: fetching and cleaning.

### 4.4.1. Step 1: Fetching

A scraper issues an HTTP GET request to the target URL and receives the HTML response. For static pages — HTML that is fully present in the server response — this is straightforward. For dynamic pages that load content via JavaScript after the initial page load, the scraper must use a headless browser (a browser controlled programmatically, without a visible screen) to wait for JavaScript execution before reading the DOM.

FreightPath's carrier portals are partially dynamic: the rate table is loaded by a JavaScript call after the page renders. Their scraper uses a headless browser, waits two seconds after page load for the table to appear, then reads the DOM.

### 4.4.2. Step 2: Cleaning

Raw HTML contains far more than readable content. A typical page mixes article text with navigation menus, cookie consent dialogs, social sharing buttons, advertising slots, and footer boilerplate. If all of this text is passed to the chunker, retrieval quality suffers dramatically — a query about freight rates might return a chunk containing "Accept all cookies to continue browsing

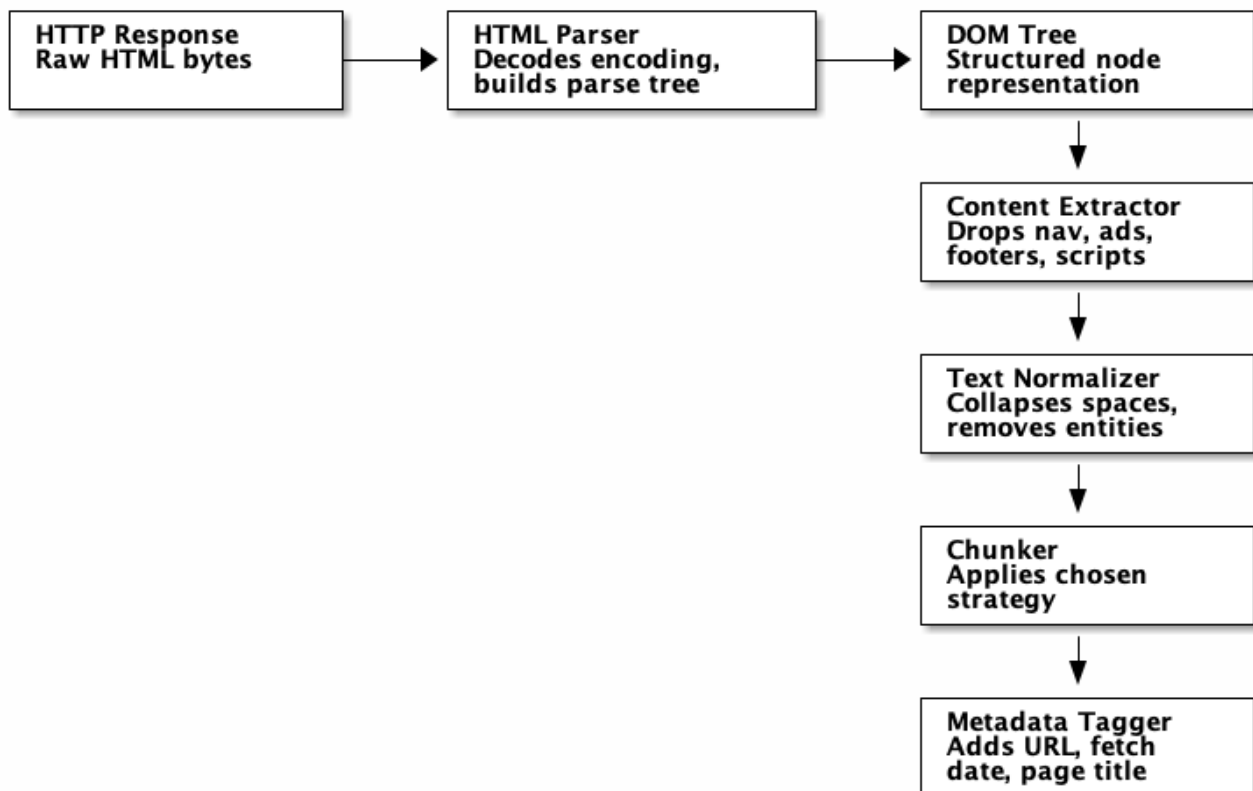
our site."

Cleaning removes non-content elements. The approach depends on how consistent the target pages are:

- **Selector-based cleaning:** if the target site has consistent HTML structure, the scraper selects only the known content element (such as `<main>` or a specific CSS class) and discards everything else. This is fast and precise but brittle — if the site redesigns, the selector breaks.
- **Density-based cleaning:** tools like Trafilatura and Mozilla Readability analyze the entire page and identify the section with the highest ratio of meaningful text to HTML markup. This is more robust to site redesigns but occasionally misidentifies the content area.

For FreightPath's carrier portals, selector-based cleaning is used because the portals are controlled environments with stable HTML structure.

#### 4.4.3. From Raw HTML to Clean, Chunked Text



#### 4.4.4. Politeness and Rate Limiting

Web scraping at scale must respect the target server. Sending thousands of requests in rapid succession can overload a server and may violate the site's terms of service. Production scrapers implement a delay between requests (typically one to five seconds), respect the `robots.txt` file that sites use to declare which pages should not be crawled, and cache responses so the same page is not re-fetched unnecessarily.

FreightPath's scraper runs nightly, fetches only pages that have changed since the last crawl

(detected via HTTP cache headers), and waits three seconds between requests to each carrier portal.

#### 4.4.5. Web-Specific Metadata

Chunks derived from web pages carry additional metadata fields:

- **url**: the exact URL from which the content was fetched
  - **fetch\_date**: the date and time the page was retrieved (web content changes; the fetch date records what version was ingested)
  - **page\_title**: the HTML `<title>` element content
  - **http\_status**: the response code (useful for detecting redirect chains or soft 404 errors)
-

# Chapter 5. How It Solves the Problem

Returning to FreightPath and Priya’s question: "What is the maximum payload for a Class 7 truck under the updated 2023 federal guidelines, and how does our current carrier agreement with Apex Logistics handle overweight penalties?"

Before the RAG pipeline existed, Priya spent four hours searching manually. After the pipeline is built and populated, here is what happens when she submits the same question.

Her query is converted to a vector. The retrieval system searches the vector store and applies metadata filters (`document_category = "federal_regulation" OR "carrier_agreement"; document_date >= "2022-01-01"`). It returns three chunks in under 200 milliseconds:

- Chunk A: from `federal_class7_guidelines_2023.pdf`, Page 14, Section 3.2 — contains the 33,000-pound payload limit
- Chunk B: from `apex_logistics_agreement_2024.docx`, Page 9, Clause 8.4.2 — contains the overweight penalty schedule
- Chunk C: from `apex_logistics_agreement_2024.docx`, Page 9, Clause 8.4.3 — contains the dispute resolution procedure for penalty charges

The language model reads these three chunks and generates a precise answer with citations. Total time: under ten seconds.

| Dimension                       | Before RAG Pipeline                                                 | After RAG Pipeline                                            |
|---------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------|
| Time to answer Priya’s question | 4 hours (manual search across 8,000 files)                          | Under 10 seconds                                              |
| Source traceability             | None — Priya had to remember where she found each fact              | Every fact cites file name, page number, section              |
| Currency of information         | Depends on whether Priya knows which file is most recent            | Metadata filter excludes outdated documents automatically     |
| Scalability                     | Linear — 40 officers x 4 hours each day                             | Sub-linear — same pipeline answers all queries simultaneously |
| Cross-format search             | Impossible in a single pass — each format requires a different tool | Unified — PDF, DOCX, HTML, CSV, web all searchable together   |
| New document onboarding         | Manual distribution and reading                                     | Ingest through pipeline in minutes, immediately searchable    |

The four components of this chapter — loading, chunking, metadata, and web scraping — are what make this transformation possible. Without robust loading, text from some document types would be garbled or missing. Without thoughtful chunking, retrieved segments would be too long, too short, or split across meaningful boundaries. Without metadata, the system could not filter by source or surface citations. Without web scraping, live rate information from carrier portals would be stale.

# Chapter 6. Summary

Building a RAG knowledge base begins long before any user types a question. The preparation pipeline must handle the full diversity of enterprise document formats — PDFs, DOCX files, HTML pages, CSVs, Markdown, and live websites — extracting plain text from each while discarding format-specific structure that adds noise without meaning. That text must then be divided into chunks of appropriate size using a strategy that matches the document’s nature: fixed-size for uniform efficiency, sentence-based for semantic precision, recursive for structured documents with headings and clauses.

Every chunk must carry metadata that answers three critical questions: where did this come from, when was it produced, and where exactly in the source document does this passage live. This metadata is not administrative overhead — it is the mechanism by which a retrieval system can filter irrelevant sources, surface citations, and maintain accuracy as documents age and are superseded. Web content adds the additional considerations of dynamic rendering, content extraction from noisy HTML, and the discipline of polite crawling. Together, these four capabilities convert an unorganized archive of thousands of documents into a queryable knowledge base that can answer specific questions in seconds with full source attribution.

---

# Chapter 7. Quick Self-Check Questions

## 7.1. Foundational Questions

1. **Why can a PDF that was created by scanning paper require a different loading approach than a PDF exported from a word processor?**

*Hint: Think about what is physically stored inside each type of PDF file. Is it characters, or is it something else?*

2. **What is the purpose of the overlap parameter in fixed-size chunking, and what problem does it address?**

*Hint: Imagine a sentence that falls exactly on a chunk boundary. Which chunk does it belong to? How might overlap help?*

3. **Name two metadata fields that would be especially important for a compliance team that needs to verify whether information is current.**

*Hint: Information can become outdated in two ways — the original document may be old, or the copy in your system may not have been refreshed recently. Which fields track each?*

4. **Why is it inadequate to simply load a CSV row as a plain comma-separated string for use in a RAG chunk?**

*Hint: Imagine reading the string "7, 26001, 33000, federal" without knowing what columns these numbers correspond to. Is it understandable on its own?*

5. **What is the difference between a statically rendered HTML page and a dynamically rendered one, and why does this distinction matter for a web scraper?**

*Hint: Think about when the content you want to read actually appears in the browser versus when the server sends its response.*

## 7.2. Intermediate Questions

1. **FreightPath adds a new document type to its pipeline: Excel spreadsheets containing multi-sheet financial reports. The first sheet is a table of carrier costs; the second is a narrative summary written in prose cells. What loading and chunking considerations arise that would not apply to a simple CSV?**

*Hint: Consider how Excel differs from CSV in structure. How might the narrative prose sheet require a different chunking strategy than the tabular cost sheet?*

2. **A retrieval system returns a chunk that contains accurate information from a 2019 carrier agreement that was superseded by a 2024 renegotiation. The metadata fields `source_file` and `ingestion_date` are both present. What additional metadata field, and what retrieval-time filter, would have prevented this outdated chunk from surfacing?**



*Hint: The ingestion\_date tells you when you added it to the system, not when the document itself was issued. What other date would you want to capture?*

3. **A team uses sentence-based chunking on a document that contains a numbered list of 20 safety procedures, where each item is a single short sentence of about 8 words. The target chunk size is 256 tokens. What is a potential retrieval problem with the chunks this produces, and how might recursive chunking or a custom rule address it?**

*Hint: Consider what each 8-word sentence looks like in isolation without the list heading that explains what all 20 items are about.*

4. **Two engineers debate whether to include the full document title and section heading in the chunk text itself, versus keeping them only as metadata fields. What are the arguments for each approach, and are they mutually exclusive?**

*Hint: Think about what the embedding model actually sees when it converts a chunk to a vector. Does it see metadata fields, or only the text content of the chunk?*

5. **A website scraped for FreightPath's RAG system undergoes a redesign six months after the initial crawl. The CSS class name used in the selector-based cleaner changes, and subsequent crawls ingest navigation text instead of rate content. Propose a monitoring or validation strategy that would detect this problem before it significantly pollutes the knowledge base.**

*Hint: What properties of legitimate rate-table chunks could you check programmatically—for example, their length, the presence of expected keywords, or their structural similarity to previously known-good chunks?*